

Software Architecture and Design Specification

Project: Vehicle Parking System

Version: 1.0

Authors: Team 10 – Pandi Snehitha, Pramiti Ragavendra Udupa, Ramitha R

Date: 05-10-2026

Status: Draft – submitted for review

Course: Software Engineering (UE24CS341A) | Semester 5 | Section F | PES University, Bengaluru

Team Profile

Sl. No	Name	SRN (USN)	PRN
1	Pandi Snehitha	PES1UG24CS315	PES1202400290
2	Pramiti Ragavendra Udupa	PES1UG24CS331	PES1202400373
3	Ramitha R	PES1UG24CS368	PES1202403685

Revision History

Version	Date	Author	Change Summary
0.1	02-10-2026	Pramiti Ragavendra Udupa	Initial architecture outline, component list and pattern selection
0.2	04-10-2026	Pramiti Ragavendra Udupa	Added sequence diagrams, API design, security architecture and traceability
1.0	05-10-2026	Team 10	Reviewed by team; baseline version for Part-1 submission

Approvals

Role	Name	Signature/Date
Author (Architecture & Design)	Pramiti Ragavendra Udupa	
Reviewer (Team Member)	Pandi Snehitha	
Reviewer (Team Member)	Ramitha R	
Course Faculty		

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Vehicle Parking System (VPS) developed by Team 10 as the Software Engineering mini-project. It translates the requirements captured in the VPS Software Requirements Specification (SRS) into components, interfaces, data stores, security controls and interaction flows, and records the rationale behind each major architectural decision so that the system can be implemented, tested and maintained consistently.

1.2 Scope

The Vehicle Parking System is a web-based system for managing parking slots and tracking vehicles in a multi-level parking facility (for example a campus, mall or office parking lot). This document covers:

- User registration, login and role-based access for Drivers, Parking Attendants and Administrators.
- Real-time slot availability by parking lot, level and vehicle type (two-wheeler / four-wheeler).
- Advance slot reservation (booking), cancellation and automatic expiry of no-show bookings.
- Vehicle entry (check-in) and exit (check-out) with automatic slot allocation and release.
- Parking fee calculation based on configurable tariffs, payment recording and receipt generation.
- Vehicle tracking – locating a parked vehicle by registration number and viewing parking history.
- Administration of lots, levels, slots and tariffs, plus occupancy and revenue reports.

Out of scope: physical boom-barrier / sensor hardware control, ANPR (automatic number-plate recognition) cameras and a native mobile app. The architecture keeps extension points for these (see Section 4.6).

1.3 Audience

Team 10 developers, QA / test engineers preparing the Software Test Plan, course faculty and evaluators, security reviewers, and future maintenance teams.

1.4 Definitions

Term	Definition
VPS	Vehicle Parking System – the system described in this document
Slot	A single marked parking space identified by a code such as L1-A-07 (Level-Row-Number)
Booking	An advance reservation of a slot by a Driver for a specific time window
Parking Session	The period between a vehicle's check-in (entry) and check-out (exit)

Term	Definition
Tariff	Pricing rule per vehicle type: base fee, hourly rate, grace period and daily cap
MERN	MongoDB, Express.js, React.js, Node.js – the chosen technology stack
SPA	Single Page Application (the React front end)
REST / API	Representational State Transfer / Application Programming Interface
JWT	JSON Web Token – signed token used for stateless authentication
RBAC	Role-Based Access Control (roles: DRIVER, ATTENDANT, ADMIN)
ADR	Architecture Decision Record
STRIDE	Threat model: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	Transport Layer Security (HTTPS / WSS encryption)
ODM	Object Document Mapper (Mongoose for MongoDB)
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals and constraints, stakeholders, the UML component diagram, component responsibilities, the chosen architecture pattern, technology stack and data stores, risks, traceability to SRS requirements, the security architecture (STRIDE threat model), the deployment view and the architecture decision records. Section 4 describes the design: UML sequence diagrams for key flows, REST API design for each component, error handling / logging / monitoring, UX design and open issues. Section 5 contains the glossary, references and tools. Requirement IDs used here (VPS-F-xx, VPS-NF-xx, VPS-SEC-xx) are the same IDs used in the SRS and the Test Plan, so a reader can trace a requirement from the SRS → architecture component → design element → test case.

2.2 Related Documents

- **SRS** – Vehicle Parking System Software Requirements Specification v1.0 (Team 10).
- **STP** – Vehicle Parking System Software Test Plan v1.0 (Team 10).
- **RTM** – Requirements Traceability Matrix (SRS Appendix C / Test Plan Section 13).
- **Problem Statement** – SE Mini-Project Problem Statements, Section F, Team 10: "Vehicle Parking System – a system for managing parking slots and vehicle tracking" (JavaScript MERN Stack / Python Flask-Django).

3. Architecture

3.1 Goals & Constraints

Goals:

- **Correctness of allocation:** a slot is never double-booked or double-occupied, even under concurrent requests.
- **Real-time visibility:** slot status changes are pushed to all connected clients within 3 seconds.
- **Performance:** 95% of API requests complete within 2 seconds with 100 concurrent users.
- **Security:** authenticated, role-based access; passwords hashed; all traffic over TLS.
- **Availability:** 99% uptime during facility operating hours; no data loss on a single DB node failure.
- **Maintainability:** modular layered code base, $\geq 70\%$ unit-test coverage on the service layer.
- **Usability:** responsive UI usable on desktop and mobile browsers; check-in in ≤ 3 clicks for attendants.

Constraints:

- Technology stack fixed by the problem statement: JavaScript MERN stack (MongoDB, Express.js, React.js, Node.js).
- Academic project timeline (sprint-based, Part-1 due 07-10-2026) and a three-member team.
- Free / open-source tools only; payment integration in gateway test (sandbox) mode – no real money is moved.
- No dedicated parking hardware; entry/exit is recorded by the attendant through the web console.
- Personal data (name, phone, email, vehicle number) must be protected in line with the Digital Personal Data Protection Act, 2023 (India).

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Drivers (end users)	Find and reserve a free slot quickly, transparent fees, locate their vehicle, privacy of personal data
Parking Attendants	Fast check-in / check-out, correct fee calculation, simple console, works on a tablet
Administrator / Facility Owner	Configure lots, slots and tariffs; accurate revenue and occupancy reports; auditability
Developers (Team 10)	Modularity, clear interfaces, testability, a stack the team knows
QA / Testers	Traceable requirements, deterministic APIs, test data and environments
Course Faculty / Evaluators	Adherence to IEEE-style documentation, sound architecture rationale, security coverage

3.3 Component (UML) Diagram

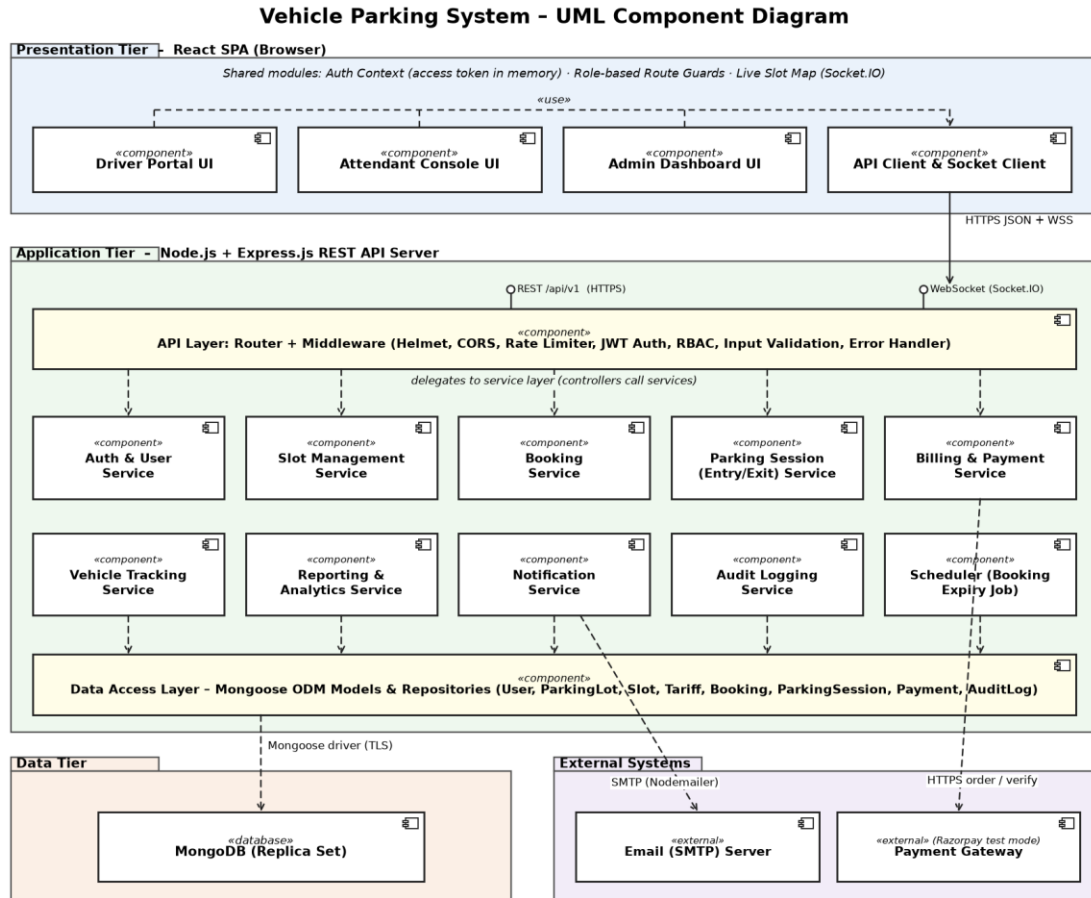


Figure 1: UML component diagram of the Vehicle Parking System (three-tier, layered)

3.4 Component Descriptions

Component	Responsibility	Key Interfaces
Driver Portal UI	Registration/login, live slot map, booking and cancellation, my bookings, vehicle locator, payment history	Uses API Client
Attendant Console UI	Vehicle check-in/check-out, fee display, mark payment, print/show receipt, live occupancy view	Uses API Client
Admin Dashboard UI	Manage lots/levels/slots, tariffs, users/roles; occupancy & revenue reports; audit log view	Uses API Client
API Client & Socket Client	Axios wrapper adding JWT, refresh-token handling, central error mapping; Socket.IO subscription to slot updates	HTTPS REST, WSS
API Layer (Router + Middleware)	Routes /api/v1/*, Helmet security headers, CORS whitelist, rate limiting, JWT verification, RBAC checks, request validation (Joi), central error handler	Provides REST /api/v1 and Socket.IO endpoints
Auth & User Service	Register, login, logout, token refresh, password hashing (bcrypt), account lock-out, user/role management	IAuthService
Slot Management Service	CRUD for lots, levels and slots; availability query; atomic slot state transitions (AVAILABLE → RESERVED → OCCUPIED → AVAILABLE; MAINTENANCE)	ISlotService
Booking Service	Create/cancel bookings with overlap check in a MongoDB transaction; booking code generation; booking validation at	IBookingService

Component	Responsibility	Key Interfaces
	entry	
Parking Session Service	Vehicle check-in (allocate slot), check-out (release slot), active session lookup	ISessionService
Billing & Payment Service	Tariff lookup, fee calculation, payment recording (Cash / UPI / Card via gateway test mode), receipts, refunds for cancelled prepaid bookings	IBillingService, Payment Gateway API
Vehicle Tracking Service	Find a vehicle by registration number (current lot/level/slot, entry time), vehicle parking history	ITrackingService
Reporting & Analytics Service	Occupancy %, peak hours, revenue by day/lot/vehicle type, CSV export	IReportService
Notification Service	Booking confirmation/expiry e-mails (Nodemailer/SMTP) and real-time 'slot:update' events (Socket.IO)	INotifier
Audit Logging Service	Append-only log of security- and money-relevant events (login, booking, check-in/out, payment, admin changes)	IAuditLog
Scheduler	node-cron job every minute: expire no-show bookings 15 min after start time and release the slot; send reminders	Internal
Data Access Layer	Mongoose schemas, validation, indexes and repositories for all collections	Mongoose models
MongoDB	Persistent store (replica set for transactions and fail-over)	MongoDB wire protocol over TLS
Payment Gateway (external)	Creates payment orders and returns signed confirmations (test mode)	HTTPS REST
SMTP Server (external)	Delivers e-mail notifications	SMTP + STARTTLS

3.5 Chosen Architecture Pattern and Rationale

Pattern: Layered (three-tier client–server) architecture, with an MVC-style Router → Controller → Service → Repository layering inside the back end, plus event-driven push (publish/subscribe over Socket.IO) for live slot status.

- **Presentation tier:** React SPA – renders views; holds no business rules other than input hints.
- **Application tier:** Express.js API – routing/middleware layer, service (business-logic) layer, and data access layer. Each layer only calls the layer directly below it.
- **Data tier:** MongoDB replica set accessed only through Mongoose models.

Rationale:

- Clear separation of concerns makes each layer independently testable (Jest unit tests on services with mocked repositories; Supertest on routes).
- Matches the MERN stack mandated by the problem statement and the team's skills, keeping the learning curve low within the sprint timeline.
- Security controls (authentication, RBAC, validation, rate limiting) are centralised in one middleware layer instead of being repeated per feature.
- Socket.IO publish/subscribe gives near real-time slot updates without clients polling the server.

Alternatives considered and rejected:

- **Microservices:** rejected – independent deployment and scaling are not needed at the scale of one facility; would add service discovery, distributed transactions and DevOps overhead the team cannot justify.
- **Monolithic server-rendered MVC (e.g. Django templates):** rejected – a SPA gives a smoother live slot map and lets the same REST API serve a future mobile app.
- **Serverless functions:** rejected – persistent WebSocket connections and scheduled jobs are awkward and cold-starts hurt the 2 s response target.

3.6 Technology Stack & Data Stores

Layer / Concern	Technology
Front end	React 18, Vite, React Router, Axios, Socket.IO client, Tailwind CSS
Back end	Node.js 20 LTS, Express.js 4, Socket.IO 4, node-cron
Database	MongoDB 7 (3-node replica set – required for multi-document transactions), Mongoose 8 ODM
Security	jsonwebtoken (JWT), bcrypt (cost 12), Helmet, cors, express-rate-limit, Joi validation, TLS 1.2+ via Nginx
Payments / E-mail	Razorpay API in test mode; Nodemailer over SMTP + STARTTLS
Logging & Monitoring	Winston (JSON logs, daily rotation), Morgan (HTTP access log), PM2 monitoring, /health endpoint
Testing	Jest, Supertest, React Testing Library, Postman/Newman, JMeter (load)
DevOps	Git + GitHub (team repository), GitHub Actions CI (lint + test), PM2, Nginx on Ubuntu 22.04

Data stores (MongoDB collections):

Collection	Key Fields	Indexes / Notes
users	_id, name, email, phone, passwordHash, role (DRIVER ATTENDANT ADMIN), vehicles[], failedAttempts, lockUntil, createdAt	unique(email)
parkingLots	_id, name, address, levels[{levelNo, name}], operatingHours, isActive	unique(name)
slots	_id, lotId, level, code, vehicleType (TWO_WHEELER FOUR_WHEELER), status (AVAILABLE RESERVED OCCUPIED MAINTENANCE), currentSessionId	unique(lotId, code); (lotId, vehicleType, status)
tariffs	_id, lotId, vehicleType, baseFee, hourlyRate, gracePeriodMin, dailyCap, effectiveFrom	(lotId, vehicleType, effectiveFrom)
bookings	_id, bookingCode, userId, slotId, vehicleNo, startTime, endTime, status (CONFIRMED CHECKED_IN CANCELLED EXPIRED COMPLETED), createdAt	unique(bookingCode); (slotId, startTime, endTime, status)
parkingSessions	_id, vehicleNo, vehicleType, slotId, bookingId?, entryTime, exitTime, durationMin, status (ACTIVE CLOSED), attendantId	(vehicleNo, status); partial unique: one ACTIVE session per vehicleNo
payments	_id, sessionId, amount, mode (CASH UPI CARD), gatewayOrderId, status (PENDING PAID FAILED REFUNDED), receiptNo, paidAt	unique(receiptNo)
auditLogs	_id, actorId, action, entity, entityId, ip, timestamp, details	(timestamp); append-only (no update/delete API)

Fee rule used by the Billing service: chargeable minutes = max(0, duration – gracePeriodMin); hours = ceil(chargeable minutes ÷ 60); fee = baseFee + hourlyRate × hours, limited to dailyCap per 24 hours. Tariff values are configured by the Administrator, never hard-coded.

3.7 Risks & Mitigations

#	Risk	Mitigation
R1	Race condition – two drivers book the same slot at the same moment (double booking)	Overlap check and insert inside a MongoDB transaction; atomic findOneAndUpdate on slot status; 409 Conflict returned to the loser
R2	Database node failure causes downtime or data loss	3-node replica set with automatic fail-over; daily mongodump backups kept 7 days
R3	Payment gateway unavailable	Cash mode always available; online payment marked PENDING and retried; session can be closed after manual confirmation by attendant
R4	Stale slot map on clients (WebSocket disconnects)	Socket.IO auto-reconnect; full availability refresh on reconnect; 30 s polling fallback
R5	No-show bookings block slots	Scheduler expires bookings 15 min after start time and releases the slot
R6	Credential stuffing / brute-force on login	Rate limiting, account lock after 5 failed attempts, bcrypt hashing, generic error messages
R7	Team skill gaps / schedule slip in sprints	Layered modules assigned per member, shared coding standards, CI checks on every push

3.8 Traceability to Requirements

Each SRS requirement is mapped to the architectural component(s) that realise it, the design elements in Section 4 and the planned test cases in the Software Test Plan.

Req. ID	Requirement (summary)	Architecture Component(s)	Design Reference	Test Case ID
VPS-F-01	User registration & login	Auth & User Service, API Layer	Seq. Diagram 3; API /auth/*	TC-AUTH-01..03
VPS-F-02	Role-based access (Driver / Attendant / Admin)	API Layer (RBAC middleware)	Sec. 3.9; API role column	TC-AUTH-04
VPS-F-03	View real-time slot availability	Slot Mgmt Service, Notification Service	Seq. Diagram 1; GET /slots/availability	TC-SLOT-01
VPS-F-04	Reserve a slot for a time window	Booking Service	Seq. Diagram 1; POST /bookings	TC-BOOK-01, 02
VPS-F-05	Cancel a booking	Booking Service, Billing & Payment	PATCH /bookings/{id}/cancel	TC-BOOK-03
VPS-F-06	Vehicle entry (check-in) & slot allocation	Parking Session Service, Slot Mgmt	Seq. Diagram 2; POST /sessions/entry	TC-SESS-01
VPS-F-07	Vehicle exit (check-out) & fee calculation	Parking Session, Billing & Payment	Seq. Diagram 2; POST /sessions/{id}/exit	TC-SESS-02, TC-BILL-01
VPS-F-08	Payment & receipt	Billing & Payment Service	Seq. Diagram 2; POST /payments/{id}/pay	TC-PAY-01
VPS-F-09	Vehicle tracking / locate vehicle	Vehicle Tracking Service	GET /vehicles/{vehicleNo}/location	TC-TRK-01
VPS-F-10	Manage lots, slots and tariffs	Slot Mgmt Service, Billing (tariffs)	POST/PUT /admin/slots, /admin/tariffs	TC-ADM-01
VPS-F-11	Occupancy & revenue reports	Reporting & Analytics Service	GET /reports/*	TC-RPT-01
VPS-F-12	Notifications & auto-expiry of bookings	Notification Service, Scheduler	Sec. 3.4 Scheduler	TC-NOTIF-01

Req. ID	Requirement (summary)	Architecture Component(s)	Design Reference	Test Case ID
VPS-NF-01	95% of requests ≤ 2 s at 100 concurrent users	API Layer, indexes in DAL	Sec. 3.6 indexes; 4.4 monitoring	TC-PERF-01
VPS-NF-02	Slot status pushed to clients within 3 s	Notification Service (Socket.IO)	Seq. Diagrams 1, 2	TC-PERF-02
VPS-NF-03	99% availability in operating hours	MongoDB replica set, PM2 cluster	Sec. 3.10 Deployment	TC-REL-01
VPS-NF-04	No double booking under concurrency	Booking Service (transactions)	Seq. Diagram 1 (alt)	TC-BOOK-04
VPS-NF-05	Responsive, accessible UI	React SPA	Sec. 4.5 UX Design	TC-UI-01
VPS-SEC-01	Authentication with hashed passwords & lock-out	Auth & User Service	Seq. Diagram 3; Sec. 3.9	TC-SEC-01
VPS-SEC-02	Authorisation – RBAC on every endpoint	API Layer (RBAC middleware)	Sec. 3.9; Sec. 4.3	TC-SEC-02
VPS-SEC-03	Data in transit encrypted (TLS 1.2+)	Nginx, MongoDB TLS	Sec. 3.10	TC-SEC-03
VPS-SEC-04	Input validation against injection / XSS	API Layer (Joi, sanitisation)	Sec. 4.4	TC-SEC-04
VPS-SEC-05	Audit trail of critical actions	Audit Logging Service	Sec. 3.9; 4.4	TC-SEC-05

Note: requirement and test-case IDs are kept identical across the SRS, this SAD and the STP so that the RTM (SRS Appendix C) can be completed with the Architecture and Design reference columns above.

3.9 Security Architecture

Security objectives: (1) Confidentiality – protect user credentials and personal data (contact details, vehicle numbers); (2) Integrity – booking, session and payment records cannot be altered by unauthorised users; (3) Availability – the service resists abuse such as brute-force and request flooding; (4) Accountability – all critical actions are traceable to an authenticated user.

Security controls by layer:

- **Transport:** HTTPS/WSS only (TLS 1.2+ terminated at Nginx), HSTS header; MongoDB connections use TLS and authentication.
- **Authentication:** bcrypt-hashed passwords (cost 12); short-lived access JWT (15 min, HS256, secret in environment variable) and refresh token (7 days) in an HttpOnly, Secure, SameSite=Strict cookie; logout revokes the refresh token.
- **Authorisation:** RBAC middleware checks the role on every route; ownership checks ensure a Driver can only read/cancel their own bookings.
- **Input handling:** Joi schemas for every request body/query; express-mongo-sanitize to block NoSQL operator injection; React output encoding against XSS.
- **Abuse protection:** express-rate-limit (100 req/min/IP general; 5 login attempts/15 min); account lock-out after 5 failed logins; request body size limit 100 kB.
- **Secrets & configuration:** secrets only in .env files excluded from Git; separate keys for dev/test; payment gateway in test mode.
- **Logging:** no passwords, tokens or full card/UPI details in logs; audit log collection is append-only.

Threat modelling (STRIDE):

Threat	Example in VPS	Mitigation
Spoofing	Attacker logs in as another driver or attendant	bcrypt passwords, JWT signature verification, lock-out, rate limiting
Tampering	Modifying fee amount or booking time in the request	Fee always computed server-side from tariff; Joi validation; gateway signature verification for payments
Repudiation	Attendant denies collecting cash for a session	Audit log records actor, action, timestamp and IP for check-in/out and payments; receipts with unique numbers
Information disclosure	Leaking user phone numbers or other users' bookings	TLS everywhere; ownership checks; API returns only required fields; generic error messages
Denial of service	Flooding the booking or login API	Rate limiting, payload size limits, PM2 cluster, Nginx connection limits
Elevation of privilege	Driver calling /admin endpoints	RBAC middleware on every route; role taken only from verified JWT, never from request body

3.10 Deployment View

Vehicle Parking System - Deployment View

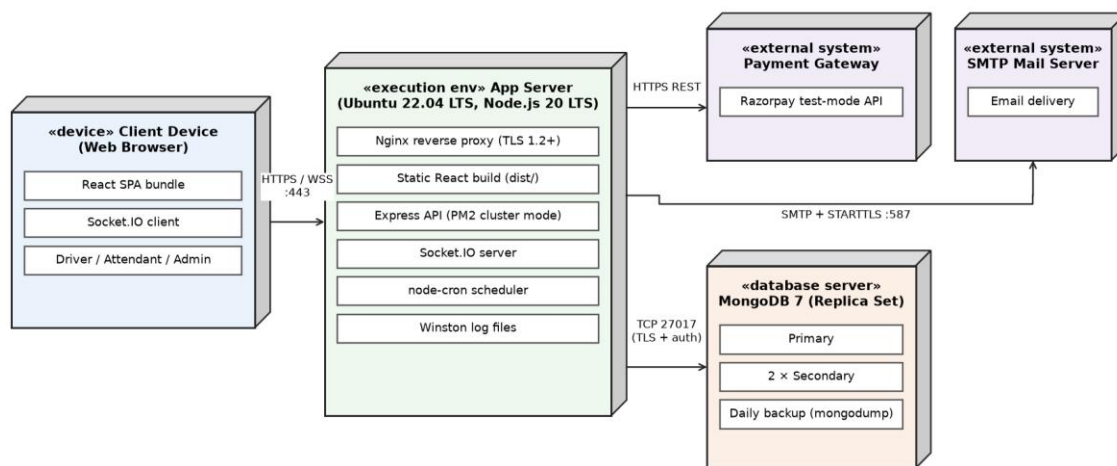


Figure 2: Deployment view of the Vehicle Parking System

The React build is served as static files by Nginx, which also acts as the TLS-terminating reverse proxy for the Express API and Socket.IO. The API runs under PM2 in cluster mode (one worker per CPU core; Socket.IO uses sticky sessions). MongoDB runs as a three-member replica set. For development, each member runs the stack locally with Docker-based MongoDB.

3.11 Architecture Decision Records (ADRs)

ADR	Decision	Status / Consequence
ADR-01	Use layered three-tier MERN architecture (not microservices)	Accepted – simple deployment and testing; scale vertically / with PM2 cluster if needed
ADR-02	Use MongoDB transactions (replica set) for booking creation	Accepted – guarantees no double booking; requires replica set even in development
ADR-03	Use Socket.IO for live slot status instead of client	Accepted – lower latency and server load;

ADR	Decision	Status / Consequence
	polling	polling kept as fallback
ADR-04	Stateless JWT access tokens + HttpOnly refresh cookie	Accepted – API servers stay stateless; refresh tokens stored hashed so they can be revoked
ADR-05	Compute all fees on the server from stored tariffs	Accepted – prevents client-side tampering; tariff changes need no code change

4. Design

4.1 Design Overview

The back end is organised by feature module, and every module follows the same internal layering:

```

server/src/
  routes/      auth, slot, booking, session, payment,
               vehicle, report, admin (*.routes.js)
  middleware/  authenticate, authorize(roles), validate(schema),
               ratelimiter, errorHandler
  controllers/ thin: parse request -> call service -> HTTP response
  services/    AuthService, SlotService, BookingService,
               SessionService, BillingService, TrackingService,
               ReportService, NotificationService, AuditService
  models/      User, ParkingLot, Slot, Tariff, Booking,
               ParkingSession, Payment, AuditLog (Mongoose)
  jobs/        bookingExpiry.job.js (node-cron)
  sockets/     slot.events.js
client/src/
  pages/ (driver, attendant, admin), components/, api/,
  context/AuthContext, hooks/useSlotSocket

```

Design principles applied: single responsibility per service; controllers contain no business logic; services depend on repository interfaces (easy to mock in unit tests); all money values are stored as integers in paise to avoid floating-point errors; all timestamps stored in UTC and displayed in IST.

Key state machines:

- **Slot:** AVAILABLE → RESERVED (booking confirmed) → OCCUPIED (check-in) → AVAILABLE (check-out); AVAILABLE ↔ MAINTENANCE (admin); RESERVED → AVAILABLE (cancel / expiry).
- **Booking:** CONFIRMED → CHECKED_IN → COMPLETED; CONFIRMED → CANCELLED (by driver) or EXPIRED (by scheduler).
- **Payment:** PENDING → PAID | FAILED; PAID → REFUNDED (cancelled prepaid booking).

4.2 UML Sequence Diagrams

Three sequence diagrams are given for the most important flows of the system: slot reservation, vehicle entry/exit with payment, and user login.

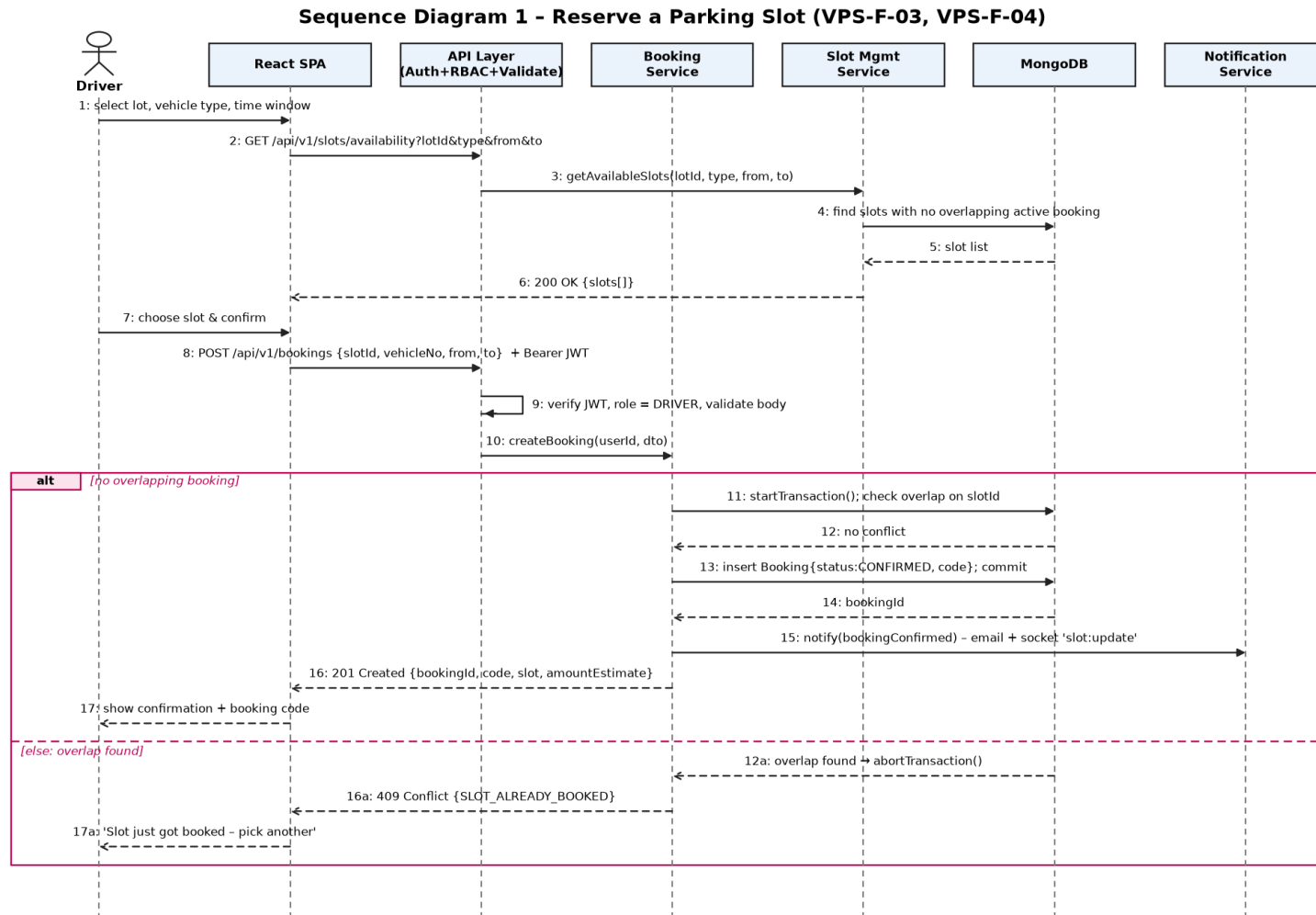


Figure 3: Sequence diagram – reserve a parking slot (including concurrent double-booking case)

Flow 1 – Reserve a slot: the driver queries availability, selects a slot and submits the booking. The Booking Service checks for an overlapping active booking and inserts the new booking inside one MongoDB transaction; if another driver booked the same slot first, the transaction is aborted and 409 Conflict is returned. On success, the Notification Service e-mails a confirmation and broadcasts 'slot:update' to all clients.

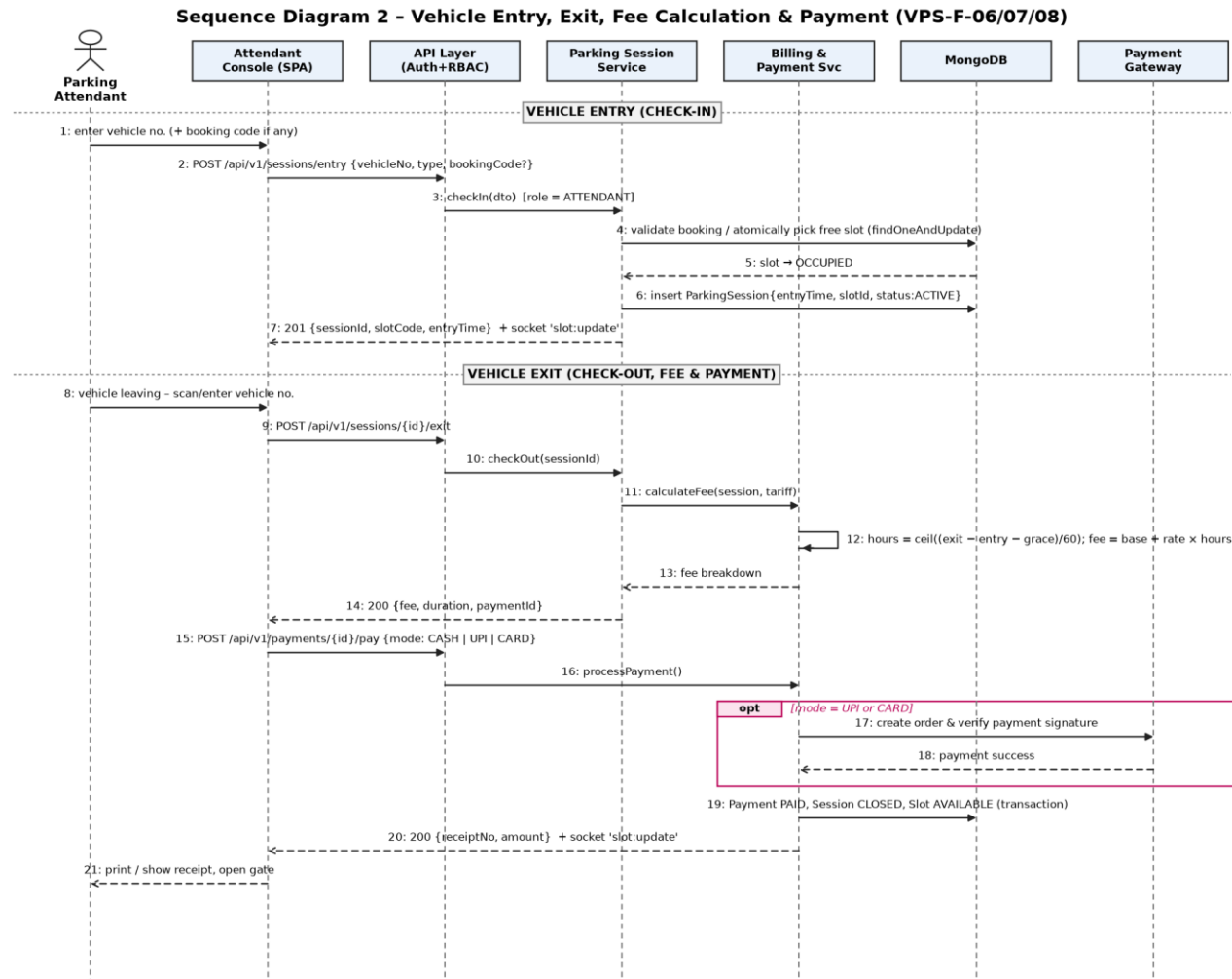


Figure 4: Sequence diagram – vehicle entry, exit, fee calculation and payment

Flow 2 – Entry and exit: at entry the attendant records the vehicle; the session service validates the booking code (if any) or atomically allocates a free slot of the right type and opens a parking session. At exit the Billing service computes the fee from the tariff, the payment is recorded (online modes go through the gateway in test mode), and the session is closed and the slot released in a single transaction.

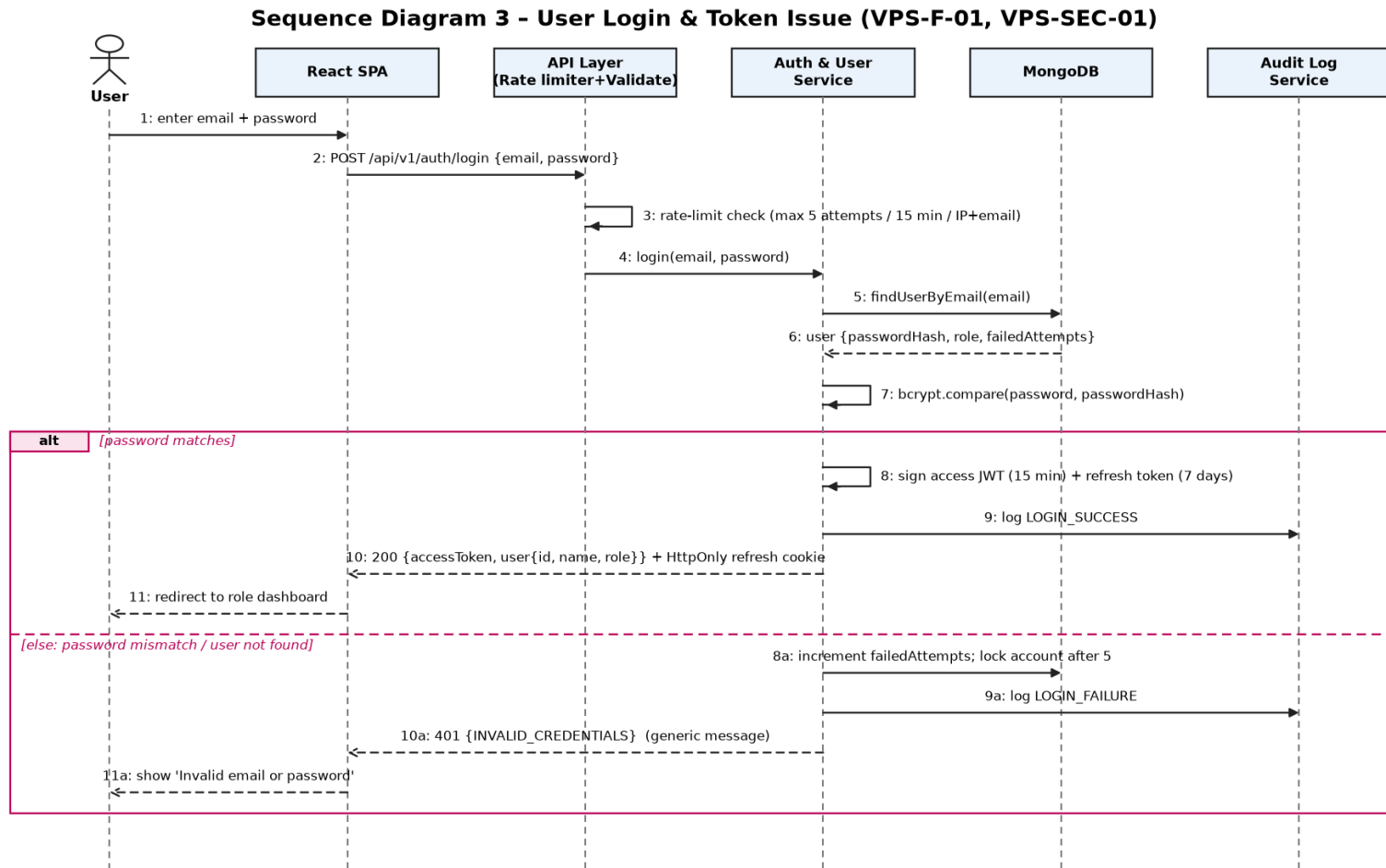


Figure 5: Sequence diagram – user login and token issue

Flow 3 – Login: the request is rate-limited and validated, the password is checked with bcrypt, and on success short-lived access and refresh tokens are issued. Failures increment a counter that locks the account after five attempts; both outcomes are written to the audit log.

4.3 API Design

Interface definitions for the main components. Base URL: `https://<host>/api/v1`. All requests and responses use JSON; protected endpoints require the header `Authorization: Bearer <accessToken>`. Errors use the common error format defined in Section 4.4.

Component	Endpoint	Method	Role	Purpose
Auth	/auth/register	POST	Public	Create a driver account
Auth	/auth/login	POST	Public	Authenticate, issue tokens
Auth	/auth/refresh	POST	Cookie	Issue new access token
Auth	/auth/logout	POST	Any	Revoke refresh token
Slot Mgmt	/slots/availability	GET	Any	Free slots by lot, type and time window
Slot Mgmt	/admin/slots	POST / PUT / DELETE	Admin	Manage slots (soft delete)
Booking	/bookings	POST	Driver	Create booking
Booking	/bookings/me	GET	Driver	List own bookings
Booking	/bookings/{id}/cancel	PATCH	Driver	Cancel own booking
Parking Session	/sessions/entry	POST	Attendant	Vehicle check-in
Parking Session	/sessions/{id}/exit	POST	Attendant	Vehicle check-out, compute fee
Billing & Payment	/payments/{id}/pay	POST	Attendant / Driver	Record or initiate payment
Billing & Payment	/admin/tariffs	GET / PUT	Admin	View / update tariffs
Vehicle Tracking	/vehicles/{vehicleNo}/location	GET	Driver (own) / Attendant	Locate parked vehicle
Vehicle Tracking	/vehicles/{vehicleNo}/history	GET	Driver (own) / Admin	Parking history
Reporting	/reports/occupancy, /reports/revenue	GET	Admin	Reports (date range, CSV export)

Detailed interface definitions:

(a) Booking Service – Create Booking

```
Endpoint: /api/v1/bookings
Method: POST Role: DRIVER
Request: { "slotId": "66f1c0...", "vehicleNo": "KA01AB1234",
           "startTime": "2026-10-10T09:00:00+05:30",
           "endTime": "2026-10-10T12:00:00+05:30" }
Response: 201 Created
          { "bookingId": "6702ab...", "bookingCode": "VPS-7K3Q9",
            "slotCode": "L1-A-07", "status": "CONFIRMED",
            "estimatedFee": 11000 } (amounts in paise)
Errors: 400 VALIDATION_ERROR (bad vehicle no. / end <= start /
                               start time in the past)
        401 UNAUTHORIZED (missing or expired token)
        403 FORBIDDEN (role is not DRIVER)
        404 SLOT_NOT_FOUND
        409 SLOT_ALREADY_BOOKED (overlapping booking exists)
        422 VEHICLE_TYPE_MISMATCH (slot type != vehicle type)
```

(b) Parking Session Service – Vehicle Check-in

Endpoint: /api/v1/sessions/entry
Method: POST Role: ATTENDANT
Request: { "vehicleNo": "KA01AB1234", "vehicleType": "FOUR_WHEELER",
"bookingCode": "VPS-7K3Q9" } (bookingCode optional)
Response: 201 Created
{ "sessionId": "6703cd...", "slotCode": "L1-A-07",
"entryTime": "2026-10-10T09:04:12+05:30" }
Errors: 400 VALIDATION_ERROR
404 BOOKING_NOT_FOUND
409 VEHICLE_ALREADY_INSIDE (ACTIVE session exists)
409 LOT_FULL (no AVAILABLE slot of this type)
410 BOOKING_EXPIRED

(c) Parking Session / Billing – Vehicle Check-out

Endpoint: /api/v1/sessions/{sessionId}/exit
Method: POST Role: ATTENDANT
Request: { } (exit time is taken from the server clock)
Response: 200 OK
{ "sessionId": "6703cd...", "durationMin": 135,
"fee": { "baseFee": 2000, "hourlyRate": 3000,
"hours": 3, "total": 11000 },
"paymentId": "6703ef...", "paymentStatus": "PENDING" }
Errors: 404 SESSION_NOT_FOUND
409 SESSION_ALREADY_CLOSED

(d) Billing & Payment Service – Pay

Endpoint: /api/v1/payments/{paymentId}/pay
Method: POST Role: ATTENDANT (cash) / DRIVER (online)
Request: { "mode": "CASH" }
or { "mode": "UPI", "gatewayPaymentId": "...", "signature": "..." }
Response: 200 OK
{ "receiptNo": "RCPT-20261010-00042", "amount": 11000,
"status": "PAID" }
Errors: 400 INVALID_PAYMENT_MODE
402 PAYMENT_FAILED (declined / signature check failed)
409 ALREADY_PAID

(e) Auth Service – Login

Endpoint: /api/v1/auth/login
Method: POST Role: Public (rate limited)
Request: { "email": "user@example.com", "password": "*****" }
Response: 200 OK
{ "accessToken": "<JWT>",
"user": { "id": "...", "name": "...", "role": "DRIVER" } }
Set-Cookie: refreshToken=...; HttpOnly; Secure; SameSite=Strict
Errors: 400 VALIDATION_ERROR
401 INVALID_CREDENTIALS (same for unknown e-mail / wrong pwd)
423 ACCOUNT_LOCKED (5 failed attempts; retry in 15 min)

(f) Vehicle Tracking Service – Locate Vehicle

Endpoint: /api/v1/vehicles/{vehicleNo}/location
 Method: GET Role: DRIVER (own vehicles) / ATTENDANT / ADMIN
 Response: 200 OK
 { "vehicleNo": "KA01AB1234", "lot": "Main Block", "level": "L1",
 "slotCode": "L1-A-07", "status": "PARKED",
 "entryTime": "2026-10-10T09:04:12+05:30" }
 Errors: 403 FORBIDDEN (vehicle not registered to this driver)
 404 VEHICLE_NOT_PARKED

Real-time events (Socket.IO namespace /live): server → client 'slot:update' {slotId, lotId, status, updatedAt}; client joins room 'lot:<lotId>' to receive only updates for the lot being viewed.

4.4 Error Handling, Logging & Monitoring**Error handling:**

- All errors flow to one Express error-handling middleware; services throw typed `AppError(code, httpStatus, message)`.
- Standard error response body: { "error": { "code": "SLOT_ALREADY_BOOKED", "message": "This slot is already booked for the selected time.", "requestId": "c1f2..." } }.
- Unexpected errors return 500 INTERNAL_ERROR with a generic message – stack traces are never sent to the client.
- Validation errors (400) list the failing fields so the UI can highlight them.
- Transactions (booking, check-out + payment) are rolled back on any failure, leaving slot state consistent.
- Front end: Axios interceptor maps error codes to user-friendly toasts; 401 triggers a silent token refresh, then logout if refresh fails.

Logging:

- Winston structured JSON logs (levels: error, warn, info, debug) with timestamp, requestId, userId and route; daily rotation, kept 14 days.
- Morgan HTTP access log (method, URL, status, response time).
- Sensitive data never logged: passwords, tokens, payment signatures; vehicle numbers masked in debug logs.
- Audit log (MongoDB auditLogs) for login, booking, cancellation, check-in/out, payment and admin changes.

Monitoring:

- GET /api/v1/health returns API and database status (used by PM2 / uptime checks).
- Metrics tracked: API p95 latency, error rate (5xx), failed logins per hour, current occupancy %, failed payments, booking-expiry count.

- Alerts (e-mail to admin) when error rate > 5% over 5 minutes or the database is unreachable.

4.5 UX Design

- **Driver Portal:** colour-coded live slot map per level (green = available, amber = reserved, red = occupied, grey = maintenance); booking in three steps (lot & time → slot → confirm); 'Find my vehicle' search; booking history with receipts.
- **Attendant Console:** tablet-friendly large inputs; check-in with one vehicle-number field and optional booking code; check-out shows the fee breakdown before payment; one-click cash payment and printable receipt.
- **Admin Dashboard:** slot/tariff management tables with inline edit; occupancy and revenue charts with date filters and CSV export.
- **Accessibility & responsiveness:** responsive layout (≥ 360 px mobile to desktop), WCAG 2.1 AA colour contrast, status shown by text/icon as well as colour, keyboard navigation and ARIA labels.
- **Feedback:** loading indicators for operations > 300 ms, confirmation dialogs for cancel/delete, clear error messages from the standard error codes.

4.6 Open Issues & Next Steps

- Integration with ANPR cameras for automatic number-plate capture at entry/exit gates.
- IoT slot sensors to update slot occupancy automatically instead of by attendant action.
- Dynamic (peak-hour) pricing and monthly passes.
- Native / PWA mobile app with QR-code-based entry using the booking code.
- Live payment gateway integration (currently test mode only).
- Next steps (sprints): set up repository and CI → implement Auth and Slot modules → Booking and Session modules → Billing, Tracking and Reports → testing per Software Test Plan.

5. Appendices

5.1 Glossary:

VPS, Slot, Booking, Parking Session, Tariff, MERN, SPA, REST, API, JWT, RBAC, ADR, STRIDE, TLS, ODM, ANPR, SRS, STP, RTM (see Section 1.4 for definitions). ANPR – Automatic Number Plate Recognition.

5.2 References:

- ISO/IEC/IEEE 42010:2022 – Systems and software engineering: Architecture description.
- IEEE Std 1016-2009 – Software Design Descriptions.
- IEEE Std 830-1998 / ISO/IEC/IEEE 29148 – Software Requirements Specifications.
- OWASP Top 10 (2021) and OWASP API Security Top 10 (2023).
- NIST SP 800-160 – Systems Security Engineering.

- Team 10 – Vehicle Parking System SRS v1.0 and Software Test Plan v1.0.
- PES University, SE (UE24CS341A) Mini-Project Problem Statements – Section F, and Mini-Project Deliverables Part-1.

5.3 Tools:

PlantUML / draw.io (UML diagrams), Swagger / OpenAPI 3.0 (API documentation), Postman (API testing), VS Code, Git & GitHub, MongoDB Compass, Jest, JMeter.